

Last updated: January 12, 2023

COMP 3000 (WINTER 2023) OPERATING SYSTEMS

TUTORIAL 1

Tasks/Questions

- When you have first logged in to a shell, how (i.e., using what commands?) can you find out information about the environment? Consider the following as examples. Try to find out multiple ways for each (if you can).
 - The version of your Linux distribution and the version of your Linux kernel.
Among other ways: `lsb_release -a`, `cat /proc/version`, and `uname -a`
There are also files containing such information: e.g., `/etc/os-release`, and `/usr/lib/os-release`
 - RAM, remaining/total disk space, and CPU.
Among other ways: `free`, `df`, `cat /proc/cpuinfo`
 - The name (binary path) of the current shell.
`echo $SHELL`
- Using the `man` command, find out what the following commands do: `which`, `pwd`, `who`, `whoami`, `whereis`, and `whatis`.
- Linux commands can be classified as internal (built into the shell) and external (separate program binaries). How can you tell if a specific command (e.g., `cd`) is internal or external?
 - For the program you just compiled above, what if you used `hello` to run it instead of `./hello` (and why)?
Without a pathname `hello` will be interpreted as a command (either internal or external) so the system will say "not found". The dot (`.`) means the current directory so if that `hello` file is in your home directory "`./hello`" will be equivalent to "`/home/student/hello`".
 - Figure out where at least three external commands reside on the system.
Check if the output of this is empty (then it's internal): `which <command>`
As long as it is not a program binary file, it is part of the shell functionality (the shell does what you expect from the command).
- Making your own commands: the `PATH` environment variable lists the directories the shell uses to search for external commands. Where can you find documentation on it? How can you add the current directory (whichever directory you are currently in) to `PATH`? Then, how to make that change permanent? Try to identify multiple ways. You can now revisit the question about "hello" above.
You are making your own commands when you add the path of your command binary to `$PATH`. Specifically, you can append something to the existing `PATH` variable (e.g., `/home/student`):
`export PATH=$PATH:/home/student`
To make it permanent (so that you don't need to type the command above every single time before you use it), you can get it executed automatically by either the OS or the shell. In the case of `bash`, there are files like (to name a few) `~/.bash_profile`, `~/.bash_login` and `~/.bashrc`, you can put this line in. Note: you do not need to do this to make it permanent unless you

really want to add a command.

5. Look at the permissions of the program binaries of the external commands you have just found above. Who owns them? What group are they in?
6. For those same program binaries, figure out what the permission bits mean by reading the man page of `chmod` (this is the command you could use to change those permission bits).
Refer to the explanation of permissions in the instructions. We will also discuss this in class.
7. What are the owner, group, and permissions of `/etc/passwd` and `/etc/shadow`? What are these files used for (and how did you find this out)?
These are very important files, one storing user account info, one storing the hashed passwords, which will be discussed in more detail later.
8. What does it mean to have execute permission on a directory?
Whether you can enter (`cd` into) that directory.
9. The `ls` command can be used to get a listing of the files in a directory. What options are passed to `ls` to see: the permission bits above; all the files within a directory (including hidden files)? How to make a file hidden?
You can use the options “`-l`” and “`-a`”. Any file/directory with a name starting with a dot (`.`) character is hidden.
10. Compile and run `csimpleshell.c`. How does its functionality compare to that of `bash`? List at least 3 differences.
There should be many differences as it merely does very little. Examples: 1) if you want to hit Tab to autocomplete typing a command, it will not work here. 2) It does not support operators like `&` (to run in background) and `>` (redirection). 3) It does not have syntax highlighting, i.e., all text in the same color. 4) You cannot use the up and down arrows to browse previously used commands.

Last updated: January 16, 2023

COMP 3000 (WINTER 2023) OPERATING SYSTEMS

TUTORIAL 2

Tasks/Questions

Part A

1. Looking at the `.s` file produced from `gcc -S -O2 hello.c`, do you see anything familiar and discussed in the lectures of our first week?
You can see sections like `.text` and `.rodata`.
2. Where (e.g., at which line) is it supposed to call that `printf()`? If it is not doing that, why? (you can just mention what you think/believe)
You may only see `puts()` being called, due to the compiler optimization (you are not passing any arguments which `printf()`'s format string is good at handling so `puts()` will be enough). You can avoid this by either passing an argument to `printf()` or using a compiler option like `-fno-builtin`.
3. Generate the `.s` file for [syscall-hello.c](#) as well. Observing the two `.s` files, what do you think is the key difference?
It calls the `syscall()` library call instead of `printf()`.
4. You can play around by trying to see: how a function in C is reflected in assembly; how to return from a function; how variables/literals are represented; what are **symbols** (refer to commands like `nm`); etc.
We will explain a bit in class.

Part B

1. By default, gcc-produced binaries are dynamically linked (so at runtime they will require dynamic libraries to be present on the system). To compile a binary that is statically linked (so it has no external runtime library dependencies), instead do this:
`gcc -O2 -static hello.c -o hello.static`
Then compile a dynamically linked version:
`gcc -O2 -z lazy hello.c -o hello.dynamic`
How does the size of `hello.dynamic` compare with that of `hello.static`? Why?
The static one should be much bigger than the dynamic one, since it has all required libraries included. The actual size would vary with many factors (e.g., the compiler/linker version).
2. Compile the other flavor of `hello.c`: [syscall-hello.c](#). Build one static version and one dynamic version as well.

Now considering `hello.dynamic`, `hello.static`, `syscall-hello.dynamic` and `syscall-hello.static`, see what system calls each program produces by running `strace -o sys-somename.log ./somename.static` (or `./somename.dynamic`). Which version generates more system calls? Why?

Note: system calls are saved in the log file `sys-somename.log`. Feel free to save them in a different file for each version.

Dynamically linked programs (containing fewer functions supposedly) do not necessarily generate fewer system calls. It depends on what the code does. For example, I/O-intensive functions need to talk to the OS a lot (to access resources like files/devices).

Specifically, in the context of this question, the code per se does not do much (i.e., only prints once), so offloading it to libraries or including it does not make much difference. However, loading a dynamic library into the address space involves system calls, e.g., `openat()` and `read/pread()` to read metadata from `libc.so` and `mmap()` to map the actual functions for invocation.

3. See what library calls each program produces by running `ltrace -o lib-somename.log ./somename.dynamic` (or `./somename.static`). Which version generates more library calls? Why?

The statically linked version should make no library calls by design. This is the purpose of a static build, which is to ensure that all the libraries are present and that they are the correct version, hence no such dependency issues. The output of `ltrace` can be misleading, which simply says sections describing library dependencies are not found, but expected.

4. Remember when building `hello` in Tutorial 1 you did not use `-z lazy`. Comparing `hello` with `hello.dynamic`, any difference? Why this difference?
Lazy binding leaves the actual resolution of library calls to the runtime so that you can observe them with `ltrace`. With the default “now” binding, it is done at load time, so nothing will be seen at runtime.
5. Use `ldd` to find out what dynamic library dependencies dynamic versions have. What about static versions?

Part C

1. Why are the addresses inconsistent between runs?
Simple answer: because they are just virtual addresses, determined by the OS.
Accurate answer: because of ASLR (address space layout randomization). This will be briefly discussed in class.
You can disable it (not recommended for systems in real use) with:
`echo 0 | sudo tee /proc/sys/kernel/randomize_va_space`
2. Roughly where does the stack seem to be? The heap? Code? Global variables? (hints: recall the memory image layout of a process discussed in the lecture and you can search for a more detailed one somewhere; local variables go to the stack; initialized **data** and global variables go to the data segment, data allocated at runtime go to the heap.)
Local variables are: `lmsg` and `buf`, so `&lmsg` and `&buf` (the location of them) are around the stack. The value of `buf[0]` (dynamically allocated) is in the heap. We will discuss it in a bit more detail.

3. Change each `malloc()` call to allocate more than 128K. What happens to the values of `sbrk`? Why? (Hint: use `strace`)

When the amount to allocate exceeds a certain threshold `malloc()` stops using `sbrk()` and uses `mmap()` to allocate memory instead.

Last updated: January 22, 2023

COMP 3000 (WINTER 2023) OPERATING SYSTEMS

TUTORIAL 3

Tasks/Questions

1. Compile and run [3000shell.c](#).
2. Compared to [csimpleshell](#) from Tutorial 1, what functionality improvements has 3000shell introduced? List at least two improvements (functional differences). You can mention whether you found them by reading the source code or trying both shells.
By using both shells, you can see 3000shell shows the current user in the prompt whereas csimpleshell doesn't (with just a "\$"). Also, if you look at the code, you'll see 3000shell supports additional commands like "plist" (although the important command "cd" is missing in both). Moreover, you may also notice richer functionalities of 3000shell, such as signal handling, support for stdout redirection, execution in the background, etc.
3. Pay attention to lines 229-235. How can you run a program in the background? Note: "in the background" means the shell starting a program without waiting for it to finish. So, the shell returns immediately to be ready to accept the next command.
As in bash, just use the "&" operator at the end.
4. Observe the behavior when running different programs in the background. What happens to the input and output of the program? Try this for different types of programs, such as `ls` and `bc`, or `nano` and `top`, or others of your choice. Write down your observations.
For non-interactive programs (those taking only inputs from the command line arguments), no difference would be noticed. For interactive ones (e.g., `nano` and `top`) continuously using standard input/output, they may mess up the shared standard input and standard output. If both the parent process and the child process are running (not one being paused by `wait()`), with the shared file descriptors (here mostly standard input), very likely they will interfere. Lines 229-235 decide whether to wait based on `background`. So, `background` will leave both processes running and potentially competing for shared resources (if not properly handled, which is 3000shell's case).
5. You may have trouble interacting with the shell after running programs in the background. How can you recover from such a situation? Note: you can do it in a harsh way for this question. We will see a smarter way next.
You can hit Ctrl-C to terminate both processes, if the interactive program also handles Ctrl-C. Otherwise, if the terminal is not frozen (but just losing key presses) you can try the interactive program's own way repetitively (e.g., Ctrl-X for `nano`). As the last resort, you can kill them from another terminal using the `kill` command after finding out the PID.
6. 3000shell implements a simple form of output redirection (i.e., the standard output mentioned earlier). What syntax should you use to redirect standard output to a file?

Just `>filename`. Note space after the operator cannot be handled by 3000shell.

7. Make the shell output "Ouch!" when you send it a SIGUSR1 signal.
Register for SIGUSR1 in `main()` with `sigaction()` the same way as other signals. Then, in `signal_handler()`, add an `fprintf()` if `(the_signal == SIGUSR1)`.
8. If you delete line 324 (SA_RESTART), how does the behavior of 3000shell change?
SA_RESTART is to restart/resume the system call interrupted by a signal. In the case of 3000shell, if line 324 was removed, after you send SIGUSR1, the command prompt will no longer work because the system call `read()` from the library call `fgets()` cannot be restarted, so it will wait infinitely.
9. Replace the use of `find_env()` with `getenv()`. How do their interfaces differ?
`getenv()` is neater and takes only one argument: the environment variable name. It searches in the actual environment variables instead of our maintained `*envp[]`.
10. Make `plst` output the parent process id for each process, e.g., "5123 ls (5122)". Pay attention to the `stat` and `status` files in the per-process directories in `/proc`.
`/proc/<PID>/status` is human-readable and `/proc/<PID>/stat` is intended for machines to read (you need to parse it to get the needed value).
11. If the shell gets stuck as a result of running a program in the background, how can you recover elegantly (resuming it) by sending just a signal (which signal)? Why?
Send a signal to 3000shell: SIGCHLD
Why: This is like a hack, as this signal is not designed for this purpose. What you need is to make the parent wait on the child as it normally does without the "&" being specified. From the code, there are two places the parent can wait (line 205 and line 234). Look at the function `signal_handler()`, if a signal SIGCHLD is received, line 205 will be executed, which puts the parent back to waiting.
This question shows the importance of understanding the code line by line.

Last updated: January 24, 2023

COMP 3000 (WINTER 2023) OPERATING SYSTEMS

TUTORIAL 4

Tasks/Questions

1. Compile and setup [3000userlogin](#) as described above. Create a new user account. Verify that you can use 3000userlogin to login as the new user without typing the password of the new user (without logging in beforehand). Think about why you are able to avoid typing the password and still get authenticated.
As 3000userlogin simulates the steps after authentication, “setuid root” is used to bypass authentication as root. Keep in mind that user authentication is just a user-space thing, i.e., after authentication the same setuid will be involved (so we could even say we did not bypass any authentication by replacing it with setuid, but just skipped it and directly went to setuid).
2. Why is the syntax highlighting gone (coloring characters)? What simple change (one line) can you make to have 3000userlogin preserve the colors? (Hint: check the environment variables)
**Your terminal's behavior is determined by what the terminal (emulator) thinks it can do. If you use the command `env` to check, you may notice an environment variable `TERM=xterm-256color`, which is no longer there after 3000login (aside from many others gone). So you can add one line to 3000login: `setenv("TERM", "xterm-256color", 1);`
P.S. if you check `man environ`, you'll see “TERM The terminal type for which output is to be prepared”.**
3. What is returned as the user's password by `getpwnam()` (in terms of type)? Is this what you expected? Read the manual page as you find necessary.
It is the `struct passwd`, corresponding to one line of the file `/etc/passwd`.
4. Compare the uid, gid, euid, egid when running 3000userlogin as a regular user (this means without the “chown” and “chmod” commands above), running it setuid root, and running it as root (i.e., running it from a root shell without the setuid bit set, e.g., “`sudo ./3000userlogin someuser`”). Also check to see what happens when you set the setgid bit.
**regular: uid=1000, euid=1000, gid=1000, egid=1000
setuid root: uid=1000, euid=0, gid=1000, egid=1000
root: uid=0, euid=0, gid=0, egid=0
setgid root: uid=1000, euid=1000, gid=1000, egid=0
setuid only affects euid and running as root will have all ids set to be that of root. Note that the root privilege (the special admin/super user) is only determined by uid/euid but not gid/egid (so gid/egid=0 only says you are in the group called root, not necessarily a superuser).**
5. Why does 3000userlogin change its gid before changing its uid? What happens if you switch the order of these operations?
You won't be able change to another group without euid=0 (i.e., as root).

6. Make 3000userlogin use the shell that is specified in the user's password entry (which is actually read from `/etc/passwd`). Check by setting someuser's shell to a new shell and then see if that new shell runs when you run 3000userlogin. You can change a user's shell with the `chsh` command.
`chsh` replaces the shell for the user in `/etc/passwd` with the one specified.
As 3000userlogin has hardcoded `/bin/bash`, you need to use `pw_entry->pw_shell` in the code in place of `"/bin/bash"`. Then it will always invoke what's set by `chsh`.
7. Can you set 3000shell to be a user's default shell? What changes do you have to make for `chsh` to accept 3000shell? Does anything obvious break when running 3000shell this way, and how can you change 3000userlogin to fix it?
`man chsh`
The specified shell needs to be listed in `/etc/shells`. Note: if you use `"sudo chsh -s"` to change the shell this restriction won't apply. You need to type `chsh` alone as the user you're changing the shell for (this is expected, as opposed to having another superuser doing it). You'll see "UNKNOWN" instead of the username at the prompt. Just pass the environment variable "USER" as well with `setenv()`.
8. Does a user's default shell have to be a regular shell? Could it instead be an arbitrary program? How do you know?
Just give it a try.
As mentioned, the shell is just a regular program that takes user commands for execution. In a broader sense, a GUI program (that runs other programs if the user clicks on a button or types in commands) can also be considered a shell.
9. Are the environment variables set by 3000userlogin the only environment variables that are set (visible) after you successfully login? How do you know?
3000userlogin is invoking a shell (e.g., `bash`). You can find out how that shell sets more. Environment variables can be set at any point in any program. Oftentimes, both the shell and the login program set some.
10. Note that 3000userlogin uses `environ` not `envp` (as an argument to `main()`) to access environment variables. Why not use `envp`? (Try changing the code to use `envp` and see what happens: worse or better?)
Try to think from the perspective of portability. `environ` is defined in `unistd.h` (specified in the POSIX standard), while whether the invoking program passes the `envp` argument is more uncertain. However, in this specific case, in terms of syntax highlighting, `envp` looks better as it passes `$TERM` so you'll see colors with it.

Optional/advanced tasks

1. Prompt for the user's password before logging the user in (Step 2 above). To do this, you'll need to get the right password hash and then figure out how to hash the entered password to check if it matches. Recall the format of `/etc/shadow` and you may need to learn how to use the `openssl passwd` command.
If the algorithm index is 6 (in our case):
`openssl passwd -6 -salt <salt_after_the_2nd$> <user_typed_password>`
to be matched with the value after the 3rd \$. To implement it, you need some programming.

Note: you are not required to complete this task and here is the code for your reference. Alternatively, you can also use the `crypt()` function (see: `man 3 crypt`).

Last updated: February 6, 2023

COMP 3000 (WINTER 2023) OPERATING SYSTEMS

TUTORIAL 5

Tasks/Questions (Part A)

1. Compile and run [3000test.c](#). It takes a filename as an argument and reports information on the file. Try giving it the following and see what it reports:
 - o a regular text file that exists
 - o a directory
 - o a symbolic link
 - o a device file (character or block)**It reports the number of 'a' if the file is a regular file or the link points to a regular file. The length is the logical size.**
2. Change 3000test to use `lstat()` rather than `stat`. How does its behavior change?
`lstat()` does not resolve (dereference) a symbolic link. So, it returns the inode of the link itself.
3. Modify 3000test so when it is given a symbolic link it reports the name of the target. Use `readlink(2)` (this notation describes how to access the man page, e.g., `man 2 readlink`).
First, use `S_ISLNK` to check if it's a symbolic link and then `readlink()`.
4. Are there files or directories that you cannot run 3000test on? Can you configure file/directory permissions so as to make something inaccessible to 3000test? Note that this is twofold: 1) whether it is completely inaccessible to 3000test (**nothing can be displayed**, except the error msg); 2) whether it can be accessed for the search (**no access to file content**).
This is where you can improve your understanding of the relationship between inodes and directory entries, and between the inodes and their data blocks.
The permission bits are in the inode, so denied access only applies to data blocks but not the inode.
Likewise, if the directory entry is not accessible (which is like data of the directory inode), there's no way to find the inode number, not to mention what permission bits are there.

Tasks/Questions (Part B)

1. Run `dd if=/dev/zero of=foo bs=8192 count=32K` and use the `ls` command to check it. What is the logical size of the file `foo`? How much space does it consume on disk? (Hint: Look at the `size` option to `ls`). In comparison, create another file with `dd if=/dev/zero of=foo2 bs=8192 seek=31K count=1K`. Any observations?
This is the case when logical size can be greater than physical size ("holes" in a file). The first command explicitly writes that many zero's taking up physical space. The second command skips 31K blocks writing only 1K blocks (note: the skipped blocks are still part of

the file size, e.g., if you write to the second byte, there must exist the first byte), where the file system does some optimization by only allocating 1K blocks, hence the holes.

2. Run `mkfs.ext4 foo`. Does `foo` consume any more space or less? Do the same (`mkfs.ext4 foo2`) to the other file and answer the same question.

`mkfs` creates the file system structures (e.g., the superblocks) so it writes data to the file, leading to actual allocation of space. This may trigger the optimization (so the OS realizes that there are repeated 0's), which may potentially make the file smaller (for `foo`). But for `foo2`, since it's already optimized (with holes, no wasted space), extra physical space needs to be allocated to hold what `mkfs` writes.

NOTE: on certain versions of Ubuntu, you may need to use:

3. Create any file in `/mnt` (e.g., `sudo touch test.txt`) and run `sudo mount foo /mnt`. Do you still see the file you just created?

`sudo mount -o loop foo /mnt`

As discussed, mounting over an existing directory (it has to be existing) will not affect its current content. It's just like redirection or pointing to another file system from there.

4. Run `df`. What device is mounted on `/mnt`? What is this device?

the `-o loop` explicitly tells mount to use a loopback dev

You should see the loopback device mounted on `/mnt`.

Otherwise mount will fail

5. Run `sudo umount /mnt`. What have gone away and what is back?

See 3 above. The previously existing but hidden files will be back.

6. Run `dd if=/dev/zero of=foo conv=notrunc count=10 bs=512`. How does the "conv=notrunc" change `dd`'s behavior (versus the command in question 2)?

`notrunc` means, instead of erasing an existing file first (truncating to zero), `dd` will just overwrite wherever it writes, leaving anywhere not written untouched. This way, you still have the file system created previously but have overwritten part of the primary superblock.

7. Run `sudo mount foo /mnt`. What error do you get?

Well, the file system has been corrupted in the previous step (superblock).

8. What command can you run to make `foo` mountable again? What characteristic of the file system enables this command to work? (Hint: revisit the instructions of this tutorial if you have no idea)

`fsck` finds the backup superblocks and replaces the corrupted primary one with it.

Last updated: February 7, 2023

COMP 3000 (WINTER 2023) OPERATING SYSTEMS

TUTORIAL 6

A: Getting Started

Download [3000pc.zip](#), unpack, and run make to compile 3000pc-fifo and 3000pc-rendezvous.

Try to take this opportunity to read and understand the two programs line by line.

Note that these programs take 3 arguments each:

- The number of events to process
 - The number of events to produce before the producer sleeps for 1 second
 - The number of events to consume before the consumer sleeps for 1 second
1. Run both programs with the same arguments of your choice. Do this a few times with different arguments. Do they behave the same way? Do you notice any differences? (again, no “correct” answers but just play with them and document your observations)
In addition to other differences by design (e.g., the pipe’s buffer being much bigger than the 32 words of the shared memory), an important difference is that 3000pc-rendezvous’s event sequence is less deterministic (or you can say stable) than 3000pc-fifo. You can see this almost each time you run 3000pc-rendezvous. The order the producer and the consumer finish and whether they wait vary.
 2. Repeat the above experiment, this time running each program under strace (with the -f flag to trace children too). Do you notice any difference in the system calls each program makes? Remember to ignore the lines before main() that might be distracting, e.g., start looking around and after the clone() system call.
3000pc-fifo just reads/writes from/to the pipe descriptors to feed/consume words, so you see a lot of calls to read() and write() but with a simple pattern. By contrast, 3000pc-rendezvous involves multiple calls to futex(), which is the system-call implementation of those pthread mechanisms (you can ignore write() for printf()).
 3. You can see there is "#define QUEUESIZE 32" in both programs. Are they actually used? Why (not)?
Apparently, 3000pc-rendezvous relies on this value to determine the size of the shared memory maintained by ourselves. By contrast, 3000pc-fifo just makes use of the OS-maintained pipe via the pipe() system call and read()/write(), with no need for QUEUESIZE.

B: Producer/Consumer with Pipes

1. Examine the source code of 3000pc-fifo. Pay attention to the call to pipe(pipefd) on line 192 and make the connection with what has been discussed in the lecture (also look at the man page for pipe(2)). Explain the following:

- a. How does the consumer receive words to consume?
By reading from the read end of the pipe. As mentioned, the `pipe()` system call returns an array of two file descriptors, one for the read end and one for the write end.
 - b. How does the producer generate and send words to the consumer?
By writing to the write end of the pipe.
 - c. Why is the call to `srandom(time(NULL))` on line 169 necessary?
Otherwise, the generated sequence will always be the same if `random()` is not seeded properly (e.g., always seeded with the same value).
2. Replace the call to `srandom(time(NULL))` on line 169 with `srandom(42)`. What differences do you notice in `3000pc-fifo`'s behavior?
The same as above. `srandom(1)` (not seeded is equivalent to seeded with 1 or any constant value) vs. `srandom(42)`.

C: Producer/Consumer with Shared Memory

1. Examine the source code of `3000pc-rendezvous`. Explain the following:
 - a. What are a few ways that `3000pc-rendezvous` is different from `3000pc-fifo`?
You can have your own observations (e.g., IPC differences, with/without concurrency issues, determinism, etc.).
 - b. What does the call to `mmap()` on line 347 do (compared to our previous uses of `mmap()`)? How did you figure this out?
Refer to the summary of `mmap()` uses in the lecture. In this case, it's used to create a shared memory region between two processes. Check the man page of `mmap()`.
 - c. How does the producer notify the consumer that the queue is no longer empty?
See the additional explanation at the end.
 - d. How does the consumer notify the producer that the queue is no longer full?
See the additional explanation at the end.
2. What arguments can you provide to make the producer wait for the consumer? Hint: Check the size of the queue.
As long as the number of events is greater than the queue size and you slow down the consumer, the producer needs to wait.
3. What arguments can you provide to make the consumer wait for the producer? (You can disregard the wait at the very beginning as the queue is initially empty)
Different from the producer, the consumer relies on available items generated by the producer. So, as long as the producer is slower (e.g., by waiting more) than the consumer, the consumer needs to wait.
4. Another student tells you that the difference between processes and threads is that processes never share memory, while threads do. Is this statement correct or incorrect? How can the behavior of `3000pc-rendezvous` help justify your answer?
Apparently, we are using shared memory between processes.
5. Change the calls to `pthread_mutexattr_setpshared()` and `pthread_condattr_setpshared()` (lines 293 and 298) to take 0 instead of 1. How does the behavior of `3000pc-rendezvous` change? Does anything break?
Things will get stuck, because that makes the mutex and condition variable not accessible between processes (even if they share the memory where the mutex and variable reside). See

the man page.

Additional explanation for `3000pc-rendezvous`:

There are three mechanisms involved, 1) the per-entry semaphore (`sem_t lock`), which ensures that an entry is not manipulated by both parties at the same time. 2) the condition variable (`queue_nonempty/nonfull`), used to coordinate between the producer and the consumer so that they know they can stop waiting and work. 3) the mutex (`cond_mutex`), as is required by the condition variable.

There is also the fourth one, which is the Boolean variable `predicate_nonempty/nonfull` used in the predicate before each condition wait. Otherwise, you may notice a deadlock after certain number of events (usually large, e.g., 400,000). While there could be various factors causing this, one known issue is called spurious wakeup or lost wakeup. A brief explanation is: when a process yields the CPU to wait on something, it no longer runs. The OS/scheduler will wake it up when the condition is satisfied. However, waking per se might not be atomic or even reliable. So, by the time the waken process gets the chance to run, that condition may no longer hold (e.g., another process jumped in before and made a change). A predicate with a loop will ensure that the process will loop for one more iteration (hence waiting again) if spurious wakeup happens.

Last updated: March 7, 2023

COMP 3000 (WINTER 2023) OPERATING SYSTEMS

TUTORIAL 7 [\[LINK\]](#)

Tasks part A: Understanding special files

1. Try the following commands as a non-privileged user. What does each do? How do the files f1-f3 compare? How do they compare to /dev/urandom? Remember you can get output from /dev/urandom using cat or dd. If you use cat, make sure to pipe it to less!

- o `cp /dev/urandom f1` (hit Ctrl+C right away to avoid a huge file)
- o `sudo cp -a /dev/urandom f2`
- o `sudo mknod f3 c 1 9`

Note: If you see any command running for more than a few seconds, hit Ctrl+C right away, check the produced file (often it's very large) and delete it if not needed.

“-a” preserves all properties of the device node, so you are creating an identical copy (equivalent to calling `mknod`, hence the need for root). By contrast, without “-a” `cp` just tries to read from the device node (invoking the driver's `read()` function) and write to another created regular file. As `/dev/urandom` never sends an end-of-file (EOF) you will get infinite data, hence filling up your disk (that's why you must stop it right away).

2. Make named pipes using `mknod` and `mkfifo`. Use them to simulate `ls | wc` using just the `'>'` and `'<'` operators.

```
wc < mypipe & ls > mypipe
```

Here assume you have a named pipe in your current directory called `mypipe`. The `&` operator runs the first one in the background and starts the second so that both can execute at the same time (not blocked). You can also use two terminals as was done in one lecture.

This shows while internally `|` is just implemented using an unnamed pipe, you can also manually achieve the equivalent using the named pipe, which is exposed as a file (in this case `mypipe`) in the file system.

3. Use `mknod` to make a copy of your current terminal's tty - `/dev/tty` (Hint: consider how you did something similar with `/dev/urandom` above). You can name it `mytty`. Examine its characteristics using `stty --file=mytty`. Do the same for the original tty.

The main purpose here is to show you that when both the major number and the minor number are the same, you are technically operating the same device instance. Reflected from your tty, you should be able to see (from task 4 below) that disabling echo for `mytty` is equivalent to disabling echo for your original tty.

Note: the exact behavior of multiple device nodes with identical major/minor varies with device drivers. In previous tutorials, you might have seen that your current tty is also `/dev/pts/1` (which `/dev/tty` points to). But if you do the same to it, the PTMX/PTS driver won't allow it (Input/output error).

```
ls -l /dev/tty
```

```
crw-rw-rw- 1 root tty 5, 0 Mar  8 00:51 /dev/tty
```

```
sudo cp -a /dev/tty mytty or sudo mknod mytty c 5 0, to make a copy of your current tty.
```


Now, you can `stty -F mytty -echo` (this is equivalent to `stty -echo`)
To get echo back, you need to blindly type `stty echo`, and hit Enter.

4. Run `stty --help` to see what you can do with `stty`. Try disabling local echo. How does the shell behave with echo disabled? How can you restore echo without logging out and back in? Recall we did this in the class.
See above.

Tasks part B: A simple kernel module

Download the source for this [simple module](#), unpack, and build it by typing "make". You can safely ignore the warning of "Skipping BTF generation...".

1. Install the module using `sudo insmod simple.ko`. The hello message is recorded in the kernel logs. How do you view the kernel logs? How many ways are there to view them?
There are multiple ways. You can use `dmesg` to check kernel messages since the most recent boot, or view `/var/log/syslog` or `/var/log/kern.log`. If you're interested, you can try to find out how such log files are gradually archived (something, something.1, then compressed: something.2.gz, etc.).
2. Check to see that the module has been loaded. How do you do this?
`lsmod`
3. Remove the module from the kernel. What did you do?
`sudo rmmod simple`

Tasks part C: A character device kernel module (driver)

Download the source for [ones](#), a kernel module implementing a character device that outputs an unbounded string of "1"s. Build, compile, and run it as before.

1. What kernel messages does the module generate? Does it create any new files (other than `/dev/ones`)? If so, where? (Hint: search in `/sys`)
`sudo find /sys -name ones`
2. What happens when you "cat" the device `/dev/ones`? How can you limit the output in more than one way?
You will be flooding your screen.
`cat /dev/ones | less`
`cat /dev/ones | more`
`cat /dev/ones | head -100`
3. Add more `printk()`'s (where you see needed) to find out at which point which functions in `ones.c` are called and for which purpose.

Tasks part D: Getting process information from a module

Download the source [newgetpid.c](#). Build and run it as before. Hints for the questions below:

<https://elixir.bootlin.com/linux/latest/source/include/linux/sched.h>, <https://elixir.bootlin.com/linux/latest/source/arch/x86/include/asm/current.h>, <https://elixir.bootlin.com/linux/latest/source/include/linux/cred.h>

1. What type is "current"? How can you figure this out?

```
struct task_struct *
```

This is about your code reading skill. We will read it briefly in the class.

2. Modify newgetpid.c so that it creates a device file /dev/describe rather than /dev/newgetpid.

```
#define DEVICE_NAME "describe"
```

3. Make /dev/describe output the calling process's parent ID (ppid), user ID (uid), group ID (gid), effective user ID (euid), and effective group ID (egid).

Not considering namespaces:

```
snprintf(message, 150, "Your PID is %d; your PPID is %d; your UID is %d; your GID is %d; EUID is %d; your EGID is %d!\n", thepid, task_ppid_nr(current), current->cred->uid.val, current->cred->gid.val, current->cred->euid.val, current->cred->egid.val);
```

Considering namespaces:

```
snprintf(message, 150, "Your PID is %d; your PPID is %d; your UID is %d; your GID is %d; EUID is %d; your EGID is %d!\n", thepid, task_ppid_nr(current), from_kuid_munged(current_user_ns(), current->cred->uid), from_kgid_munged(current_user_ns(), current->cred->gid), from_kuid_munged(current_user_ns(), current->cred->euid), from_kgid_munged(current_user_ns(), current->cred->egid));
```

Using bpftrace to monitor kernel module events

Note: you are not required to write about your experience for this part in the submitted work, but you should still do it, which can help you understand eBPF better (see how simple but still powerful an eBPF program can be).

Download this tiny code [kmsnoop.bt](https://github.com/tytso/kmsnoop.bt) and make it executable with `chmod 755 kmsnoop.bt`.

Now, you can open a separate SSH session and run it with `sudo ./kmsnoop.bt`. It will wait for events. When you do any "insmod" or "rmmod" in the other session, you will see it outputs a line of message.

If you want to avoid opening another SSH window, you can use tmux (a terminal multiplexer) by just typing tmux and hit enter.

To create a new session: press Ctrl-B and C

To move between sessions: press Ctrl-B and N (or P)

How could this save you from doing any printk's in your kernel module code? You can also `sudo bpftrace -l | grep` for your needed kernel functions (e.g., `printk`) to hook to.

There are generally three types of "events" your bpftrace code can hook to: `tracepoints`, `kprobes` and `uprobes`. Since we are looking at kernel events, we can use either `tracepoints` or `kprobes`. They are different in the sense that `kprobes` are about "raw" access to kernel functions with original context, and hence may or may not exist (due to kernel changes). `Tracepoints` are implemented/supported by kernel developers as explicit and static attach points, hence more reliable. `Tracepoints` have well-defined data structures for you to extract. You can see details with: `sudo bpftrace -lv "[any tracepoint name]"`

We will also briefly show this in the class.

COMP 3000 (WINTER 2023) OPERATING SYSTEMS

TUTORIAL 8 [\[LINK\]](#)

Tasks part A: Getting Started

Compile `3000physicalview` and `3000memview2` using the provided Makefile (i.e., by running `make`).

Insert `3000physicalview` by running `make insert`. Confirm that the module is inserted using `lsmod`.

1. Examine the call to `copy_from_user()` and `copy_to_user()` on lines 120 and 132 of `3000physicalview.c` (as also discussed in the lecture). Consider the following:
 - How are these functions different from `get_user()/put_user()` that we have seen in the previous tutorial?
`copy_from_user()/copy_to_user()` is comparable to `memcpy()` which deals with a range of memory, whereas `get_user()/put_user()` is used for single variables (e.g., a char or int).
 - Why are these functions necessary? Couldn't we just access the userspace address directly? What could happen if we did?
If you try to use `memcpy()` to replace these four functions it *may* work. However, losing fault containment as part of the price for running code in kernel space, any invalid addresses passed into the kernel mode (e.g., null, or pointers to other processes or even the kernel's memory) will lead to kernel oops or even panic. These four functions do all the access checks and ensure that the addresses are valid.
2. `3000physicalview` exposes its API to userspace in the form of an `ioctl(2)` call. Consider the following:
 - What is an `ioctl`? How is it different from a read or write system call? Hint: check `man 2 ioctl`.
In addition to the differences in syntax, consider `ioctl()` and `read/write()` in two aspects: `read` and `write` are natural operations to a file, also applicable to special files. If they are called in-band, we can call `ioctl` an out-of-band channel, which can adjust the behavior of regular reads/writes. On the other hand, `read()` is one-way only for reading and `write()` is one-way only for writing. `ioctl()` is two-way: you can provide input and get output to fulfill any tasks.
 - How does `3000physicalview` implement its `ioctl`? What arguments does it take?
In `physicalview_ioctl()` in `3000physicalview.c`, `ioctl` takes a file descriptor (referring to the device), a command (`PHYSICALVIEW_WALK`), and an address pointing to a custom data structure (one member being the virtual address and the other being the returned physical address).
 - How does `3000memview2` call the `ioctl`? What arguments does it pass to the `ioctl`?
Looking at the call to `ioctl()` in `3000memview2.c`, the arguments passed in correspond to the three above. In particular, `PHYSICALVIEW_WALK` is defined in `3000physicalview.h`. For now, just treat it as a value agreed on by both the driver and the application to perform this translation.

3. Which function does the virtual-to-physical translation? What type is `current->mm`? How does it give you the address of the `pgd`? Recall the page table walk explained in the lecture and see how it's reflected in this function (writing it down is optional).

`get_physical()`.

`struct mm_struct`. Here you see how page tables can be per-process.

`mm_struct` contains a member pointer: `pgd_t * pgd`, pointing to its `pgd`.

Each `pgd->offset()` takes the address of its parent-level table, and returns an address to an `pte` entry corresponding to a given virtual address, all the way down to a `pte`.

4. Once you've got the page frame number (pfn), how is the physical address (phys) calculated? Describe in words.

The page frame number is retrieved from the `pte` in the last-level page table, referring to a physical page frame but not the page frame address yet. For example, if the number is `0x1234`, the address should be `0x1234000` (as the page size is 4KiB, taking 12 bits = 3 nibbles). Therefore, to get the translated physical address, we use the page frame start address (`0x1234` shifted to the left by 12 bits = `0x1234000`) + the offset within the page frame (extracting the last 12 bits from `addr`).

Tasks part B: Examining Physical Memory Mappings

1. With `3000physicalview` inserted, run `3000memview2` and examine the output. Note that it presents virtual memory addresses on the left, and physical addresses on the right. Are these mappings consistent with what you expected (e.g., in terms of patterns)?

The virtual addresses on the left demonstrates the layout of a typical memory space (as mentioned in Tutorial 2 and corresponding lectures), while the physical addresses on the right show no patterns overall.

2. Compare `3000memview2` with `3000memview` from Tutorial 2. What is similar about their code, and what is different? How similar is their output?

They both define various types of variables and print their addresses. `3000memview` uses `printf()` on their virtual addresses while `3000memview2` uses the `3000physicalview` driver to report their physical addresses as well as the virtual ones.

3. Do you notice a pattern in the virtual addresses of `buf[i]`? Is this same pattern present in the physical addresses? Why or why not?

NOTE: please use [this updated 3000memview2.c](#) to be able to see the observations discussed in class and explained below:

If you look at the last three nibbles (e.g., `6b0`) of the addresses of `buf[i]` (in the code `buf[0]` is printed), you'll see they are incrementing (continuous) and the same is seen in both virtual and physical addresses. This shows that memory is managed (allocated) in pages. No matter how the mapping is done (maybe not as you expected), the granularity is page.

The original code was printing the first byte of `buf[0]`, `buf[1]`, `buf[2]`, ... `buf[i]` which were individually allocated from `malloc()`, each being 4K. So, there addresses are not supposed to be continuous. The updated code at the end prints the first 30 bytes of `buf[0]` within its 4K.

4. Run `3000memview2` a few more times and consider the following:

- Are the virtual addresses the same or different between runs? How about physical addresses?

In general, all (both virtual and physical) addresses are changing. ASLR contributes to the change of virtual addresses, and the kernel just allocates/deallocates memory pages affected by many factors not pertaining to the process in question hence the

change of physical addresses.

- Some physical addresses don't seem to be changing between runs. Which ones? Why do you think this might be the case?
main, lmsg and gmsg. See below.

5. Force the kernel to drop the virtual memory cache using `sync && echo 3 | sudo tee /proc/sys/vm/drop_caches`. Run `3000memview2` one more time and note that the physical addresses that stayed the same previously have now changed. What do you think just happened?

What is cached and what is not:

Recall that the page cache is actually “disk cache” which stores recently accessed data from block devices, including files. Then it’s about which part of the address space is from files and which is not. Checking the memory layout (e.g., using `pmap`) you can see the program binary and various dynamic libraries are from files, while the rest is not, e.g., stack, heap, bss, etc. Among the variables of `3000memview2`, only `lmsg` and `gmsg` (in the data segment) and `main` (in the text/code segment) are from the program binary file `3000memview2`, hence cached.

Tasks part C: Using `bpftool` to Monitor `3000physicalview` and `3000memview2`

Now recall what you tried with eBPF in Tutorial 7. We can also do something similar to watch the interaction between `3000physicalview` and `3000memview2`. Note: No in-depth understanding of eBPF is expected in this course. You only need to understand what is involved and discussed. Feel free to read more if interested.

1. Use the following “one-liner” (as it is just a one-line string within the single quotation marks) in a new terminal session first and in the original one run `./3000memview2` as above:

```
sudo bpftool trace -e 'tracepoint:syscalls:sys_enter_ioctl { printf("%s: fd=%d; cmd=%d; arg=%ld \n", comm, args->fd, args->cmd, args->arg); }' | grep 3000memview2
```

What does this one-liner do? As always, you can check `man bpftool` if needed.

It hooks to an event called “`tracepoint:syscalls:sys_enter_ioctl`” which corresponds to the time when entering the `ioctl()` system call (there’s also one corresponding to its exit --- “`tracepoint:syscalls:sys_exit_ioctl`”). Then it prints information for its arguments, i.e., the involved file descriptor, command, and argument (in our case it’s the pointer to that C struct).

2. (Optional in the submission but you should do it) It seems the command above does not show useful information as the third argument is actually a pointer. So, to be able to show the two arguments `virt` and `phys` defined in `3000physicalview.h`, you will need to create a *.bt file. Refer to `/usr/sbin/*.bt` and convert the one-liner above into something like `snoop3000physicalview.bt`, so that when it runs it prints the `virt` passed in and the `phys` that is returned. The code will be posted and explained.

Note: if you choose to directly include the header file you need to create a copy and remove the `MODULE_*` macros. Otherwise, you can simply include the struct definition.

See the code at the end.

As it’s just the C syntax, you can include the C struct definition from the header file and dereference `arg` to print the two members. Note here that upon entry there’s only the virtual address passed in so you can only print the physical address upon return. But you first need to store the argument (pointer) in an eBPF map (a variable whose name starts with an “@”) as it’s unavailable in the exit event. Also, don’t forget to free up memory using `clear()`.

```

#!/usr/bin/env bpftrace
struct physicalview_memory
{
    unsigned long virt;
    unsigned long phys;
};
/* Optionally, we can add a BEGIN section to print some messages */
BEGIN
{
    printf("Tracing ioctl calls from all processes... Hit Ctrl-C to end.\n");
}
tracepoint:syscalls:sys_enter_ioctl
{
    if (comm == "3000memview2")
    {
        @param[tid] = (struct physicalview_memory *) (args->arg);
        printf("%s: fd=%d; cmd=0x%x; virt=0x%016llx, phys=0x%016llx\n",
comm, args->fd, args->cmd, ((struct physicalview_memory *)args->arg)->virt,
((struct physicalview_memory *)args->arg)->phys);
    }
}
tracepoint:syscalls:sys_exit_ioctl
{
    if (comm == "3000memview2")
    {
        printf("%s: virt=0x%016llx, phys=0x%016llx\n\n", comm,
@param[tid]->virt, @param[tid]->phys);
    }
}
END
{
    clear(@param);
}

```

COMP 3000 (WINTER 2023) OPERATING SYSTEMS

TUTORIAL 9 [\[LINK\]](#)

Tasks/Questions

The purpose of the following questions and tasks is to help you understand how containers work, which is an important artifact on top of and based on modern operating systems. The focus is on the OS building blocks – how it is made possible by the OS. Docker is just a typical and popular example to facilitate understanding.

You need to sudo for all the docker commands below.

Install Docker with `sudo apt install docker.io` if it's not already installed.

1. Use `docker images` and `docker ps` (similar to the purpose of `ps`) to see what images you have and whether any containers are running. Run your first container with `docker run hello-world`. Now, check the images and `docker ps` again, what do you see and why?
There was no image hello-world at the beginning. After running docker-run, you will see hello-world from docker-images (because it's downloaded from Docker Hub). But still we see nothing from docker-ps, as hello-world is short-running (finished quickly).
2. Look at the hello-world image above and you may wonder how the tiny-sized image forms a full container. Find out where the files are located (hint: using the command `docker inspect` and looking for paths starting with `/var/lib/docker`). What is the size of the found file? Run the file directly. What do you think is the way the tiny-sized image forms the full container?
You will see long pathnames one lower and one upper forming the layered file system. Directly running the small binary hello (found in one of the directories) will have the same effect as inside the container. The merged view of the layers will only be available inside the container.
This question may trick you into thinking there's something other than the small binary. Actually, it was statically linked (meaning it contains everything it needs) and is stand-alone.
3. `docker ps -a` will show something different. Next, let's try an image of a bit more complexity that will remain running. Try the centos (a Linux distribution) image: `docker run -d centos sleep infinity` (man `docker-run` for the meaning of the options).
docker ps -a also shows terminated containers, so you'll see hello-world. The next command is to run the centos image as a new container in the detached mode (without interacting with it via a terminal). A specific program sleep is run with the argument infinity (which will keep sleeping. See `man strtod`). This way, we can take our time to examine the running container.
4. Start a new shell: `docker exec -it <CONTAINER ID> /bin/bash` (you can find out the container ID with `docker ps -a`. Note that you can specify just the first few digits of the ID as long as it's distinctive). Explore the file system therein.
As opposed to -d, -it is to remain interactive (by redirecting the stdin to a pseudo-tty).
Next, we will examine the namespaces.

5. Compare the mount namespaces (different views of the file system structure) between the centos container and the host, using `ls -l /proc/self/ns/mnt`. Are they the same and if not, what are they (format: `mnt: [????]`)? In particular, take a look at the root mount point ("/) inside and outside the container (`mount | grep "on / "`). You can always use two sessions (one in the container and one in the host) or `tmux` as you feel convenient.

The root file system of the container will be the layered one (whose path you saw in `docker-inspect`). The namespaces will be different except for `cgroup` and `user`. The user namespace being shared also reflects the fact that with no UID mapping, the root in the container will be the root outside.

6. With the `lsns` command, you can get an overview of all namespaces. Comparing the centos container and the host, which namespaces are the same (shared) and which are different? **Here, you will have the same observations as above (all namespaces being different except for `cgroup` and `user`).**
7. Inside the container, use the command `id` to see what user you're logged in as (uid, gid and group). Then `ps -eo uid,gid,pid,command | grep sleep` inside the container and outside the container respectively. Are their outputs the same?

The user (UID) running `sleep` is the same inside and outside, which is both root (0).

8. Using the pid of "sleep" you have got outside the container above, find its pid namespace (`sudo ls -l /proc/<pid>/ns/pid`, format `pid: [????]`). Is it consistent with the `lsns` output inside the container?

This should be the same as what you saw with `lsns`.

9. From step 6, you have the individual namespaces used by the "sleep" container (no need to go inside the container again). Let's check the network namespace (`sudo nsenter -t <pid> -n ip addr`).

You can read the man page of `nsenter`, which enters the specified namespace (`-n` means the network namespace) of the target process and runs the specified command (here `ip addr`).

Note that uid remapping is not enabled by default. So the user (uid) inside the container will be the same outside (which does not consider security).

10. Now, make your own Docker container. Read the current [Dockerfile of hello-world](#). Change it in a way that it is no longer from scratch but *from hello-world*. Here we can avoid defining `csimpleshell` as the default command as it works with a terminal for interaction (then it will inherit `CMD` from `hello-world`). Build a new image called `csimpleshell` by doing the following:

```
mkdir anyname
```

```
copy your csimpleshell and the Dockerfile of hello-world into this directory
```

```
cd anyname
```

```
make necessary changes to the Dockerfile
```

```
docker build -t csimpleshell .
```

Here the changes you can consider can be to copy your `csimpleshell` binary into the container and base it on `hello-world`. Note that if you remove the `CMD` line it will just inherit from `hello-world` which can be overridden when you run it (see below).

So you can make it something like:

```
FROM hello-world
```

```
COPY csimpleshell /
```


11. Run the new csimpleshell container (e.g., `docker run -it csimpleshell /csimpleshell`). Depending on how you compiled csimpleshell, check whether the container runs or if it does whether csimpleshell works as before. Try compiling csimpleshell statically (if it was dynamic) and repeat the steps.
If your csimpleshell was dynamically linked, the error of file not found is expected, as the container image does not have the libraries. Even after you have it statically linked, inside the shell, still you cannot use any external commands (e.g., `ls`) as files are not there.
12. Edit the Dockerfile again and make it from “centos” instead of “hello-world”. Remove any reference to hello. Rebuild and run it. How is csimpleshell now? Why is there such a difference?
The new Dockerfile can be like:

```
FROM centos
COPY csimpleshell /
```

Now you should be able to do everything as before (issuing any commands like `ls`), as the centos image contains all the files needed.
Note that here it does not matter whether csimpleshell is statically linked or dynamically linked as the centos image includes everything. This is the benefit of a layered file system.